Lecture 2: The Ultimate Guide to CSS Core, Layouts, Animations & Bootstrap

1. Introduction to CSS & Browser Rendering

What is CSS? Cascading Style Sheets (CSS) is the design layer of the web. While HTML provides the skeleton, CSS provides the skin, styling, and visual layout.

The Browser Loading Process:

1. **DOM Tree:** The browser parses the HTML file.
2. **CSSOM Tree:** The browser parses the CSS rules.
3. **Render Tree:** The DOM and CSSOM are merged to determine what is visible.
4. **Layout:** The browser calculates the exact size and coordinates of each element (Box Model).
5. **Paint:** Pixels are actually painted onto the screen.

2. CSS Types: Inline, Internal, & External

External CSS is the industry standard. It keeps styling separate from structure, making your website maintainable.

```
<!-- External (Best Practice - goes in <head>) -->
<link rel="stylesheet" href="styles.css">

<!-- Internal (Used for single-page unique styles) -->
<style> p { color: blue; } </style>

<!-- Inline (Avoid if possible - overrides everything) -->
<p style="color: red;">Text</p>
```

3. Selectors & Specificity

Selectors are how we "find" the HTML elements we want to style.

- **Class (.):** Reusable across many elements (e.g., .btn).
- **ID (#):** Unique to ONE element per page (e.g., #header).

Specificity Hierarchy: `HTML Tags < Class < ID < !Important`. The more specific rule wins when conflicts happen.

```
/* Specificity Examples */
button { padding: 10px; }             /* Tag: Low Power */
.btn-primary { background: blue; }    /* Class: Medium Power */
#submit-btn { font-weight: bold; }    /* ID: High Power */
.text { color: orange !important; }  /* !important: Highest
(Overrides all) */
```

4. Colors, Backgrounds, Fonts & Lists

Basic typographic styling makes up 80% of standard web development.

```
body {
  background-color: #f0f0f0;
  color: #333333;
  font-family: 'Arial', sans-serif;
}

ul.custom-list {
  list-style-type: square; /* Can be disc, circle, or none */
  line-height: 1.8;
}
```

Assignment 1: Style an HTML Profile Page

Task: Create a profile page about yourself. Use an External CSS file to set background colors, font families, and an unordered list of hobbies with custom square bullets.

5. CSS Units & The Box Model

- **Absolute Units:** Fixed measurements like px or cm.
- **Relative Units:** Scales dynamically (e.g., %, em, rem, vh, vw).

The Box Model: Every HTML element is a box wrapped in layers: Content → Padding (inside space) → Border → Margin (outside space). Margin and Padding shorthand follows a clockwise direction: **Top, Right, Bottom, Left**.

```
.card-box {
  width: 50%; /* Relative width */
  padding: 20px; /* Space inside */
  border: 2px solid black;
  margin: 10px 15px 20px 25px; /* Clockwise: Top Right Bottom
Left */
}
```

6. Pseudo-classes & Pseudo-elements

Pseudo-classes define a special *state* (like hovering). Pseudo-elements style a specific *part* of an element.

```
.card:hover {
  background-color: lightgray;
  cursor: pointer;
}

.quote::before {
  content: "“";
  color: gray;
}
```

Assignment 2: Interactive Cards

Task: Create three HTML cards side-by-side. Use the Box Model for proper padding/margin spacing. Add a :hover effect that changes the background color and border thickness when hovered.

7. Combinators & Form Input Styling

Combinators allow targeting elements based on relationships: Descendant (`div p`), Child (`div > p`), Adjacent Sibling (`div + p`), General Sibling (`div ~ p`).

```
/* Targeting specific input types */
input[type="checkbox"], input[type="radio"] {
  accent-color: #1a73e8; /* Changes the default blue */
  transform: scale(1.5);
}
select { padding: 10px; border-radius: 5px; background: #eee; }
```

8. CSS Positions

- **Static:** Default flow.
- **Relative:** Moved relative to its normal position. Leaves a gap where it used to be.
- **Absolute:** Positioned relative to the *nearest positioned ancestor or root*. Removed from flow.
- **Fixed:** Positioned relative to the viewport. Stays during scrolling.
- **Sticky:** Toggles between relative and fixed based on scroll position.

```
.parent-box { position: relative; } /* Anchors the absolute
child */
.child-badge { position: absolute; top: 0; right: 0; }
.navbar { position: sticky; top: 0; }
```

Assignment 3: Responsive Navigation Menu

Task: Create a full-width Navbar using position: sticky. Below it, create a main content layout with a relative container holding an absolutely positioned "New!" badge in the corner.

9. Flexbox Layout (1-Dimensional)

Flexbox excels at aligning elements in a single row or column. Use `justify-content` for the main axis and `align-items` for the cross axis.

```
.flex-container {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap; /* Allows items to drop to next line */
  justify-content: space-between;
  align-items: center;
  gap: 20px;
}

.card { flex-grow: 1; } /* Expand to fill available space */
.card-first { order: -1; } /* Move to the beginning visually */
```

Assignment 4: Flexbox Cards Section

Task: Build a responsive section with 4 cards using Flexbox. Ensure they wrap onto new lines on smaller screens. Use the order property to visually rearrange the cards differently from their HTML structure.

10. CSS Grid Layout (2-Dimensional)

Grid controls both rows and columns simultaneously. Perfect for structural page layouts.

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
  gap: 15px;
}

.featured-item {
  grid-column: span 2; /* Take up 2 columns of space */
  grid-row: span 2;    /* Take up 2 rows of space */
}
```

Assignment 5: Responsive Image Gallery

Task: Create an image gallery with 6 images using CSS Grid. Make it responsive, and pick two specific images to span across 2 columns and 2 rows to create an asymmetrical dynamic layout.

11. Media Queries & Responsive Design

The industry standard is **Mobile-First Design**: Write CSS for small screens first, then adapt for larger screens using `min-width`.

```
/* Base: Mobile View (0px and up) */
.column { width: 100%; padding: 10px; }

/* Tablet View */
@media (min-width: 768px) {
  .column { width: 50%; }
}

/* Desktop View */
@media (min-width: 1024px) {
  .column { width: 33.33%; }
}
```

Assignment 6: Desktop to Mobile Conversion

Task: Take a static 3-column desktop layout. Use @media queries to ensure that on mobile devices under 768px, the layout collapses into a single-column stacked view.

12. CSS Transitions, Transforms, & Animations

```
/* 1. Smooth hover transition & 2D Transform */
.btn { transition: transform 0.3s ease; }
.btn:hover { transform: scale(1.1) translateY(-5px); }
```

```
/* 2. 3D Transform (Flip Card) */
.flip-container { perspective: 1000px; }
.flip-card { transition: transform 0.6s; transform-style:
preserve-3d; }
.flip-container:hover .flip-card { transform: rotateY(180deg); }

/* 3. Keyframe Animations */
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.05); }
  100% { transform: scale(1); }
}
.hero { animation: pulse 3s infinite ease-in-out; }
```

Assignment 7: Animated Components

Task: Create a Hero section that gently pulses using @keyframes. Create an "Animated Button" that scales up on hover. Finally, create a card component that flips over using a 3D rotateY transform.

13. Bootstrap Framework & Optimization

Bootstrap provides a pre-built 12-column Grid system and UI components to accelerate responsive development.

- **Grid System:** Uses containers, rows, and columns (col-md-4).
- **Optimization:** Always link the minified (.min.css) CDN to save load times.

```
<!-- 1. Include Minified CSS -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/boot
strap.min.css" rel="stylesheet">

<!-- 2. Bootstrap Navbar Component -->
<nav class="navbar navbar-dark bg-dark sticky-top">
  <div class="container-fluid"><a class="navbar-brand"
href="#">Brand</a></div>
</nav>
```

```
<!-- 3. Bootstrap Grid & Cards -->
<div class="container mt-5">
  <div class="row">
    <!-- Full width mobile, 1/3 width on desktop -->
    <div class="col-12 col-md-4 mb-3">
      <div class="card">
        <div class="card-body">
          <h5 class="card-title">Card</h5>
          <a href="#" class="btn btn-primary">Action</a>
        </div>
      </div>
    </div>
  </div>
</div>
```

Assignment 8: Build a Bootstrap Webpage

Task: Build a complete webpage using ONLY Bootstrap classes (write zero custom CSS). Include a sticky Navbar, a padded Hero section, and a row of 3 cards that properly collapse into a single column on mobile screens.